



How to Improve and Test the Scalability of our Application?

In this lesson, we will cover how to improve and test the scalability of our application.

We'll cover the following



- Tuning the performance of the application – Enabling it to scale better
 - Profiling
 - Caching
 - CDN
 - Data compression
 - Avoid unnecessary requests response cycles
- Testing the scalability of our application

Here are some of the standard strategies to fine-tune the performance of our web application. If the application is performance-optimized, it can withstand more traffic load with less resource consumption than an application that is not optimized for performance.

Now you might be wondering, *“Why are you talking about performance when you should be talking about scalability? Isn’t it what the lesson title says?”*



Well, the application’s *performance* is directly proportional to *scalability*. If an application is not performant, it will certainly not scale well.



These performance optimization strategies can be implemented even before the *pre-production* testing stage of the application.



Let's find out what they are.

Tuning the performance of the application – Enabling it to scale better

Profiling

Profile the hell out of your app. Run *application profiler* and *code profiler*. See what processes are taking too long and are eating up too many resources. Find out the bottlenecks. Get rid of them.

Profiling is the dynamic analysis of our code. It helps us measure the *space* and the *time complexity* of our code and enables us to figure out issues like *concurrency errors*, *memory errors* and robustness and safety of the program. [This Wikipedia resource](#) contains a good list of performance analysis tools used in the industry.

Caching

Cache wisely, and cache everywhere. Cache all the static content. Hit the database only when it is really required. Try to serve all the read requests from the cache. Use a write-through cache.

CDN

Use a *Content Delivery Network (CDN)*. Using a *CDN* further reduces the application's latency due to the proximity of the data from the requesting user.

Data compression

Compress data. Use apt compression algorithms to compress data and store data in compressed form. Since compressed data consumes less bandwidth, the data download on the client will be faster.

?

Tr

☾

Avoid unnecessary requests response cycles

Avoid unnecessary round trips between the client and server. Try to club multiple requests into one.

These are a few of the things we should bear in mind in the context of

 **educative**

Testing the scalability of our application

Once we are done with the essential performance testing of the application, it is time for capacity planning, provisioning the right amount of hardware—*compute* and *storage* power.

The right approach for testing the application for *scalability* largely depends on the design of our system. There is no standard rule for this.

Testing can be performed at both the hardware and the software level. Different services and components need to be tested—individually and collectively.

During the scalability testing, different system parameters are taken into account, such as:

- CPU usage
- Network bandwidth consumption
- Throughput
- Number of requests processed within a stipulated time
- Latency
- Memory usage of the program
- End-user experience when the system is under heavy load and so on.

In this testing phase, simulated traffic is routed to the system to study how the system behaves and scales under the heavy load. Contingencies are planned for unforeseen situations.

As per the anticipated traffic, the appropriate hardware and computational power are provisioned to handle the traffic smoothly with some buffer.

Several *load* and *stress* tests are run on the application. Tools like *JMeter* are pretty popular for running concurrent user tests on the application; if you are on the *Java* ecosystem. There are a lot of cloud-based testing tools available that help us simulate test scenarios just with a few mouse clicks.

Businesses test for scalability all the time to get their systems ready to handle a traffic surge. If it's a sports website, it prepares itself for the sports event day. If it's an e-commerce website, it makes itself ready for festival season sale and so on.

Here are a couple of good reads on the topic:

How production engineers support global events on Facebook.

How Hotstar a video streaming service scaled with over 10 million concurrent users.

In the industry, tech like *Cadvisor*, *Prometheus* and *Grafana* are pretty popular for tracking the system profile via web-based dashboards.



